TECH BIZ SECURITY · SOC ANALYST INTERNSHIP · BATCH B-503

WEEK 2 · TASK 4

Linux Essentials & Log Analysis

Submitted by	**Oscar ALIDJINOU**
Student ID	**TBS-INT-2026-2398**
Program	**SOC Analyst Internship (6 weeks)**
Company	**Tech Biz Security**
Date of submission	**24-10-2026**

01 Objective

Logs are the evidence a SOC analyst works from, and on Linux most of that evidence is plain text that I can read from the command line. This task had two goals: to become comfortable with the Linux commands used for navigating and searching, and to learn where logs live, how they are collected and managed, and how to analyse them. I completed two TryHackMe rooms, a practical failed-login search on my own Ubuntu VM, and three hands-on labs: collecting logs with rsyslog, managing them with logrotate, and parsing them into one timeline.

02 Proof of Completion

Room / activity	Duration	Tasks	Status
TryHackMe: Linux Fundamentals (Pt1)	15 min	5	Completed, room progress 100%
TryHackMe: Intro to Logs	30 min	7	Completed, room progress 100%
Practice: failed-login search on my Ubuntu VM	-	1	Done (section 06)
Lab: log collection with rsyslog	-	1	Done (section 07)
Lab: log management with logrotate	-	1	Done (section 08)
Lab: parsing and analysing logs	-	1	Done (section 09)

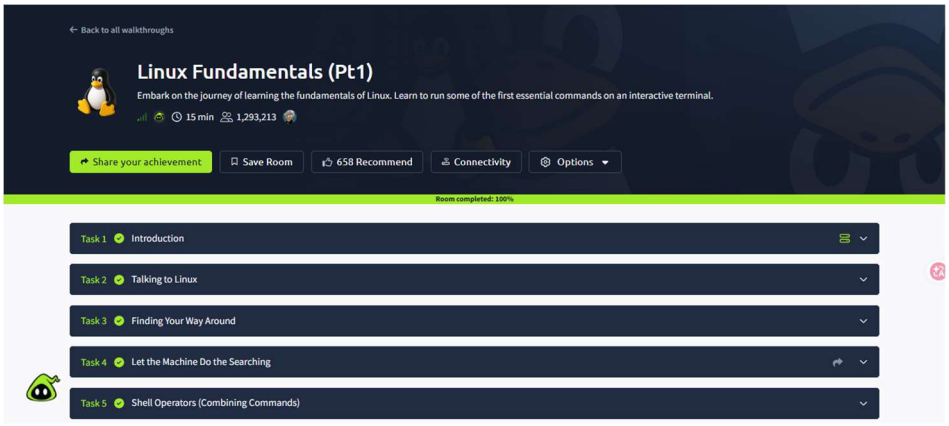


Figure 1. Linux Fundamentals (Pt1): room completed (100%).

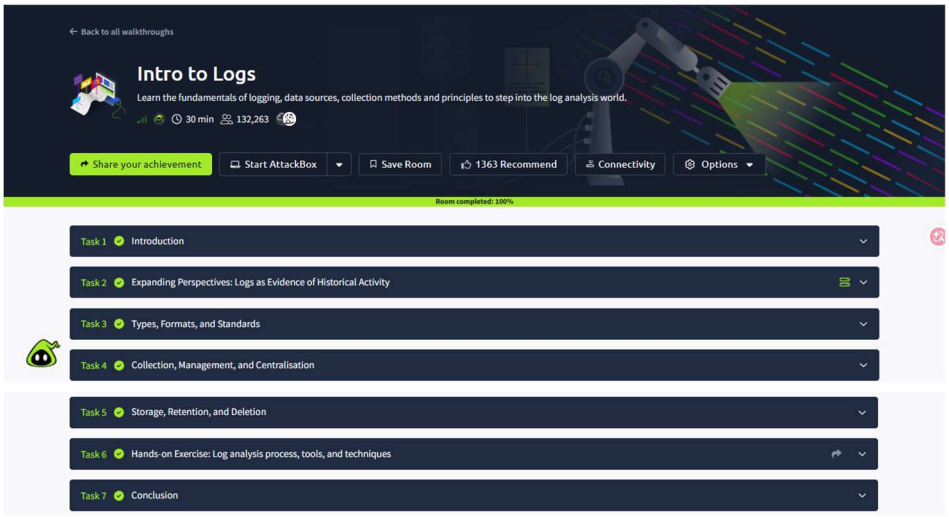


Figure 2. Intro to Logs: all tasks completed (100%).

03 Linux Commands for Navigation and Searching

Everything starts at the terminal. These are the commands I practised, grouped by what they are for.

Command	What it does	Example
whoami	Shows which user I am on the system.	`whoami`
pwd	Prints the current directory ("where am I?").	`pwd`
ls	Lists the contents of a folder; -la adds hidden files and details.	`ls -la /var/log`
cd	Moves into another directory.	`cd /var/log`
cat	Prints a whole file.	`cat note.txt`
less	Opens a long file page by page (q to quit, / to search).	`less /var/log/syslog`
head / tail	Shows the first or last lines. tail -f follows a log live.	`tail -n 20 auth.log` `tail -f auth.log`
find	Searches for files by name.	`find -name passwords.txt`
grep	Searches inside files for text.	`grep "password123" passwords.txt`
& and &&	& runs a command in the background; && runs the second command only after the first finishes.	`ls & cat file` `cmd1 && cmd2`
> and >>	Redirect output to a file: > overwrites, >> appends.	`grep Failed log > out.txt`



Figure 3. Practising ls, cd, cat, find and grep in the Linux Fundamentals room.

04 Where Logs Live in Linux

On Linux, most logs are stored under /var/log. The names differ slightly between distributions: Debian and Ubuntu use auth.log and syslog, while Red Hat-based systems use secure and messages.

Location	What it records
`/var/log/auth.log`	Authentication events on Debian/Ubuntu: SSH logins, sudo use, failed passwords, user changes.
`/var/log/secure`	The same authentication events on Red Hat, CentOS and Fedora.
`/var/log/syslog`	General system messages from services and the system (Debian/Ubuntu). Red Hat uses /var/log/messages.
`/var/log/kern.log`	Messages from the Linux kernel, such as hardware and driver events.
`/var/log/wtmp` `/var/log/btmp`	Binary records of successful logins (wtmp) and failed logins (btmp). Read them with the last and lastb commands.
`/var/log/nginx/` `/var/log/apache2/`	Web server access and error logs: every request, with its IP, URL, status code and user agent.
`/var/log/gitlab/`	Application logs for GitLab, the server I investigated in the lab (access.log is under the nginx folder).
`journalctl`	Reads the systemd journal, the log store used by modern services.
`/etc/rsyslog.d/`	Configuration for rsyslog, which decides what is logged and where it is written.
`/etc/logrotate.d/`	Configuration for logrotate, which decides how logs are rotated, compressed and deleted.

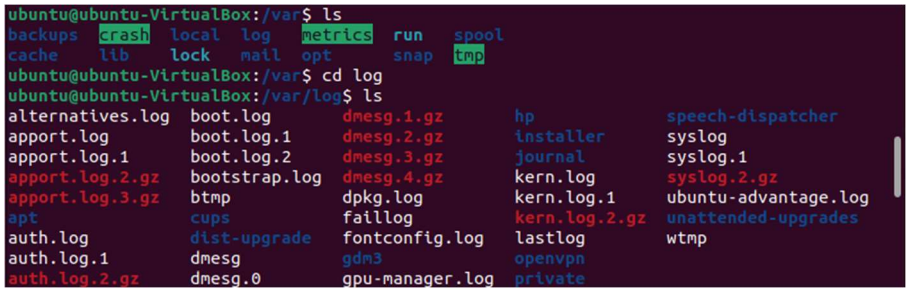


Figure 4. Listing of /var/log on my Ubuntu VM.

> **Why the folder matters**
> When an alert names a host, my first question is which log would show the activity. Login problems point to auth.log, web attacks to the web server access log, and system faults to syslog or kern.log.

05 Searching Logs with grep

grep is the analyst's fastest tool for finding the lines that matter in a file with thousands of entries. These are the patterns I use most, written for the Ubuntu authentication log.

Command	What it finds
`grep "Failed password" /var/log/auth.log`	Every failed SSH password attempt.

Command	What it finds
`grep -i "failed" /var/log/auth.log`	Any line containing "failed", ignoring upper or lower case.
`grep -c "Failed password" /var/log/auth.log`	How many failed attempts there were (a count, not the lines).
`grep "Accepted" /var/log/auth.log`	Successful logins. Useful to check right after many failures.
`grep "Invalid user" /var/log/auth.log`	Attempts to log in with usernames that do not exist.
`grep -E "Failed|Invalid" /var/log/auth.log`	Either pattern in one search (-E enables OR with |).
`grep -n "sudo" /var/log/auth.log`	Lines about sudo, with line numbers for reference.
`grep -v "CRON" /var/log/syslog`	Everything except lines that contain CRON, to cut noise.
`grep -r "Failed" /var/log/`	Searches every file in a folder and its subfolders.
`zgrep "Failed" /var/log/auth.log.*.gz`	Searches rotated, compressed logs without unpacking them.
`tail -f /var/log/auth.log | grep --line-buffered Failed`	Watches failed logins as they happen.

Counting the source of failed logins

Piping grep into other tools turns a long log into an answer. This one lists which IP addresses failed most often:

```
grep "Failed password" /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort -nr | head
```

awk picks the IP address field, sort groups identical addresses, uniq -c counts them, and the final sort puts the busiest address first. One address with hundreds of failures usually means a brute-force attack.

06 Practice: Failed-Login Search

To see a failed login in a real log, I produced one myself on my Ubuntu virtual machine and then hunted for it, following the steps on the task card.

6.1 What I did

- **Navigated to the logs:** ls /var, then cd log and ls, to find auth.log, syslog and the rotated copies.
- **Created failed logins:** I ran sudo ssh localhost and entered wrong passwords for the root account until SSH refused me (Permission denied). I did this twice.
- **Searched the authentication log:** grep "Failed" /var/log/auth.log.
- **Read the newest entries:** tail -n 5 on auth.log and on syslog.

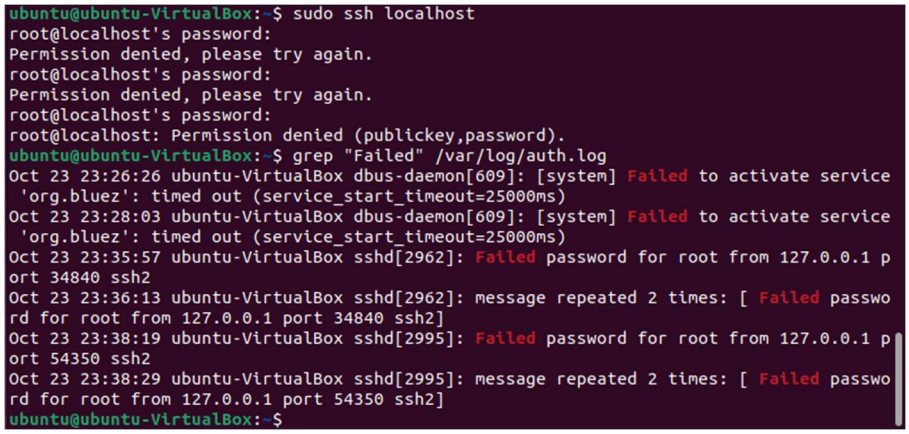


Figure 5. Failed SSH attempts for root and the grep search that finds them in auth.log.

6.2 What the results show

Time	Source	What it means
23:26:26 and 23:28:03	dbus-daemon	"Failed to activate service org.bluez". A Bluetooth service timed out. Matched by my search, but unrelated to logins.
23:35:57	sshd[2962]	First failed password for root from 127.0.0.1, port 34840 (my first test connection).
23:36:13	sshd[2962]	"Message repeated 2 times": two more failures from the same connection. Three wrong passwords in total.
23:38:19	sshd[2995]	A new process ID and port (54350) mean a second connection; again a failed password for root.
23:38:29	sshd[2995]	Another "repeated 2 times" line: three more failures, six in total across both tests.

> **Analyst assessment**
> Two SSH sessions, each with three failed passwords for root, from 127.0.0.1. Because the source is the machine itself, this is my own test and not an attack, but it has exactly the shape of a brute-force attempt. On a real server the source would be an external IP, and I would check it against threat intelligence and look for an "Accepted" line afterwards.

6.3 A lesson about false positives

My search term "Failed" was too broad. It also matched the two unrelated Bluetooth lines, which a hurried analyst might wrongly treat as login problems. A tighter search avoids the noise:

```
grep "Failed password" /var/log/auth.log
grep "sshd" /var/log/auth.log | grep "Failed"
```

6.4 The newest log entries

tail -n 5 shows the end of a log, which is where the most recent events are. In auth.log the last lines closed the story of my second test: after the failures, sshd recorded that the connection was closed before authentication finished ("preauth"), PAM reported two more authentication failures for the

root user from 127.0.0.1, and the sudo session closed. The end of syslog showed routine system housekeeping, including a message that rsyslogd was HUPed (told to reload and reopen its files), which is the signal logrotate sends after rotating logs.

```
ubuntu@ubuntu-VirtualBox:/var/log$ tail -n 5 auth.log
Oct 23 23:38:19 ubuntu-VirtualBox sshd[2995]: Failed password for root from 127.0.0.1 p
ort 54350 ssh2
Oct 23 23:38:29 ubuntu-VirtualBox sshd[2995]: message repeated 2 times: [ Failed passwo
rd for root from 127.0.0.1 port 54350 ssh2]
Oct 23 23:38:30 ubuntu-VirtualBox sshd[2995]: Connection closed by authenticating user
root 127.0.0.1 port 54350 [preauth]
Oct 23 23:38:30 ubuntu-VirtualBox sshd[2995]: PAM 2 more authentication failures; logna
me= uid=0 euid=0 tty=ssh ruser= rhost=127.0.0.1  user=root
Oct 23 23:38:30 ubuntu-VirtualBox sudo: pam_unix(sudo:session): session closed for user
 root
```

Figure 6. tail -n 5 /var/log/auth.log: the end of my failed-login test.

```
ubuntu@ubuntu-VirtualBox:/var/log$ tail -n 5 syslog
Oct 23 23:34:34 ubuntu-VirtualBox kernel: [  644.091775] workqueue: blk_mq_requeue_work
 hogged CPU for >10000us 16 times, consider switching to WQ_UNBOUND
Oct 23 23:35:42 ubuntu-VirtualBox rsyslogd: [origin software="rsyslogd" swVersion="8.21
12.0" x-pid="626" x-info="https://www.rsyslog.com"] rsyslogd was HUPed
Oct 23 23:39:06 ubuntu-VirtualBox systemd[1]: Starting Cleanup of Temporary Directories
...
Oct 23 23:39:07 ubuntu-VirtualBox systemd[1]: systemd-tmpfiles-clean.service: Deactivat
ed successfully.
Oct 23 23:39:07 ubuntu-VirtualBox systemd[1]: Finished Cleanup of Temporary Directories
```

Figure 7. tail -n 5 /var/log/syslog: routine system messages.

6.5 Anatomy of the log line

```
Oct 23 23:35:57 ubuntu-VirtualBox sshd[2962]: Failed password for root from 127.0.0.1
port 34840 ssh2
```

Part of the line	Value in my log	Why it matters
Timestamp	Oct 23 23:35:57	When it happened; needed to build a timeline.
Host	ubuntu-VirtualBox	Which machine recorded the event.
Process and PID	sshd[2962]	The SSH daemon; the PID tells separate connections apart.
Event	Failed password	What happened and whether it succeeded.
Account	root	The account targeted. For a name that does not exist, the line says "invalid user" first.
Source	127.0.0.1 port 34840	Where the attempt came from, which I can check against threat intelligence.

6.6 What I look for in real investigations

- **Many failures from one IP in a short time:** a brute-force or password-guessing attack.
- **Many different usernames from one IP:** account enumeration.
- **"Accepted password" right after a run of failures:** the attacker may have guessed correctly. This turns an alert into an incident.
- **Attempts on accounts that should never log in remotely, such as root:** worth escalating.

07 Log Fundamentals and Collection with rsyslog

7.1 What a log entry contains

A log is a record of events that happened on a system. A typical entry holds a timestamp, the system or application that wrote it, the type of event, and details such as the user or IP address involved. One entry may look unimportant, but when many are combined they answer the questions an investigator always asks: what happened, when, where, who was responsible, did it succeed, and what was the result.

Format type	Examples	Notes
Semi-structured	Syslog messages, Windows EVTX	Predictable fields plus free text.
Structured	CSV, TSV, JSON, XML, W3C Extended	Strict layout; easy to parse automatically.
Unstructured	NCSA Common and Combined Log Format (Apache and Nginx defaults)	Free text; rich but harder to parse.

Log types I should recognise: application, audit, security, server, system, network, database and web server logs.

7.2 Collection, centralisation and time

Collection means gathering logs from servers, network devices, software and databases. Centralising them in one place makes searching, alerting and incident response much faster. Clocks must also agree: if two systems disagree about the time, events cannot be put in the right order, so time synchronisation (NTP) matters.

7.3 Hands-on: collecting sshd logs with rsyslog

The lab scenario was a public-facing GitLab server. The administrator's note said the server was not yet forwarding logs to the SIEM, so they had to be collected manually. The note pointed to the GitLab web access log as the first place to look and to the logrotate configuration for retention.

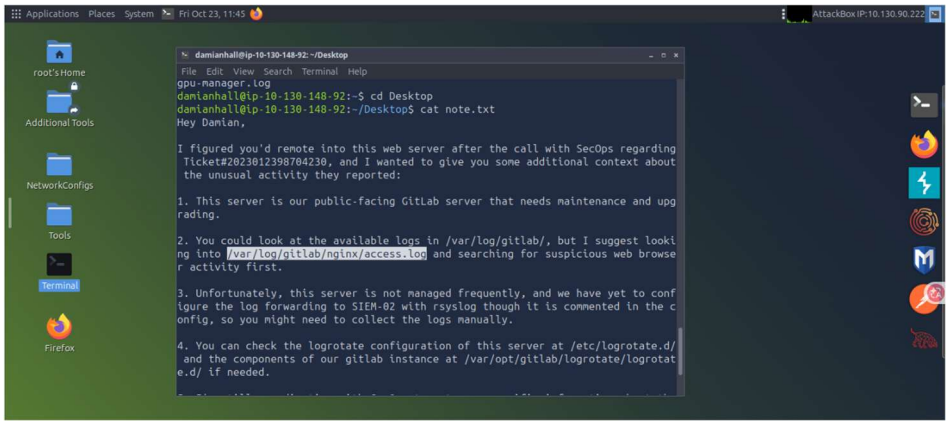


Figure 8. The administrator's note: the logs to review and the missing SIEM forwarding.

I then used rsyslog, the logging service on Ubuntu, to write all messages from the SSH daemon into their own file:

- **Checked the service:** systemctl status rsyslog showed it active (running).
- **Wrote a rule** in /etc/rsyslog.d/98-websrv-02-sshd.conf that sends every message from the program sshd to /var/log/websrv-02/rsyslog_sshd.log, with file permissions 0644.

- **Restarted rsyslog** so the rule took effect.
- **Generated a test event** by connecting with ssh localhost, which produced SSH activity for the new file to capture.

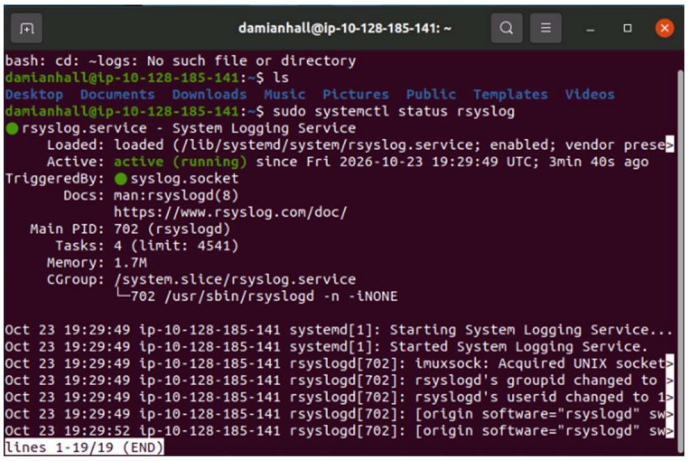


Figure 9. rsyslog service running.



Figure 10. The rsyslog rule that filters sshd messages into their own file.

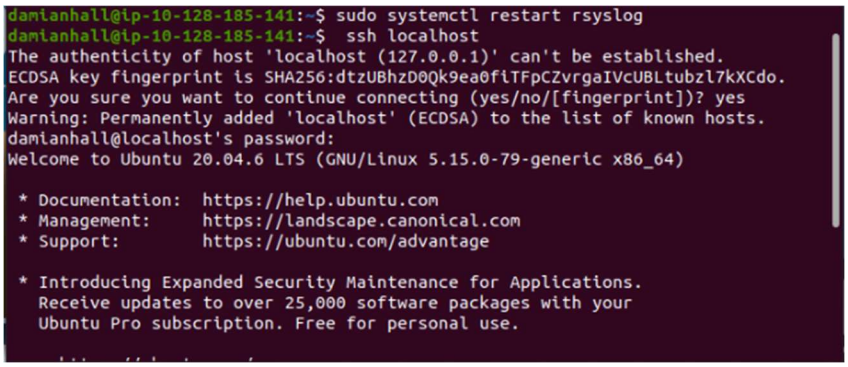


Figure 11. Restarting rsyslog and testing with an SSH connection.

> **Why a separate file helps**
> A dedicated sshd log is much quicker to search for failed logins than the full syslog, and it can be forwarded to a SIEM as its own source.

08 Log Management with logrotate

Logs cannot grow forever. Storage is finite, so old logs must be rotated (renamed and replaced), compressed and eventually deleted, while still being kept long enough for investigations. The room describes three tiers: hot storage for roughly the last 3 to 6 months, warm storage from 6 months to 2 years, and cold archived storage for 2 to 5 years. Logs should be backed up before deletion, and the policy should follow both business and legal requirements.

I wrote a logrotate configuration for the new sshd log. Each directive has a purpose:

Directive	Effect
`daily`	Rotate the log once a day.

Directive	Effect
`rotate 30`	Keep 30 rotated copies, then delete the oldest.
`compress`	Compress rotated logs with gzip to save space (.gz files).
`lastaction`	Run a script after rotation: it computes a SHA-256 hash of each rotated archive, writes the hashes to a dated file, and restarts rsyslog.



Figure 12. logrotate configuration for rsyslog_sshd.log.

I forced a rotation with sudo logrotate -f and listed the folder. New compressed archives (rsyslog_sshd.log.1.gz and .2.gz) and a dated hashes file appeared, and the hashes file listed a SHA-256 value for each archive.

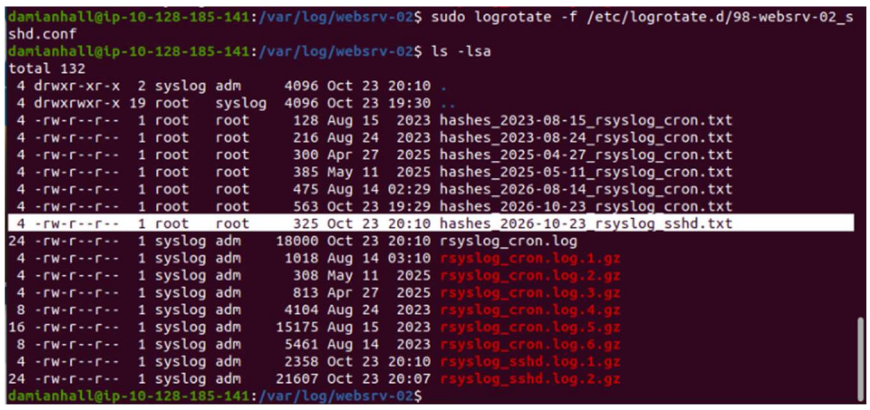


Figure 13. After forcing rotation: compressed archives and the new hashes file.

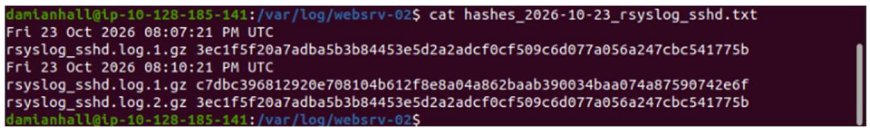


Figure 14. The hashes file records an integrity fingerprint for each rotated log.

> **Link to the CIA triad**
> Hashing each rotated log supports integrity. If someone alters an old log to hide their tracks, the stored hash will no longer match, which is also why I hash evidence files before analysing them.

09 Log Analysis: from Raw Logs to One Timeline

9.1 The analysis process

Step	Meaning	Where I used it
Parsing	Breaking raw lines into understandable fields.	awk splitting each log line.
Normalisation	Converting different formats to one standard.	Unifying the timestamp and layout across logs.
Sorting	Ordering entries by time, source or severity.	sort on the consolidated file.
Classification	Tagging entries by type or severity.	Labelling each line by its source file.
Enrichment	Adding context such as geolocation or threat intelligence.	Next step for the suspicious IP.
Correlation	Linking related entries across logs.	Reading web, SSH and cron events on one timeline.
Visualisation	Showing data as charts or in a viewer.	The Log Viewer web interface.
Reporting	Summarising findings for others.	This report.

9.2 Reading the raw logs

I first opened the unparsed logs in the open-source Log Viewer tool, which lets me read the original files with no preparation. The GitLab web access log showed a pattern right away: a single IP address, 34.253.159.159, sent request after request to a long list of different paths (for example /~tmp, /~webmaster, /buyproduct, /cachemgr.cgi) within the same second, all using the same old browser string (MSIE 6.0 on Windows NT 5.1) and all answered with a 302 redirect.

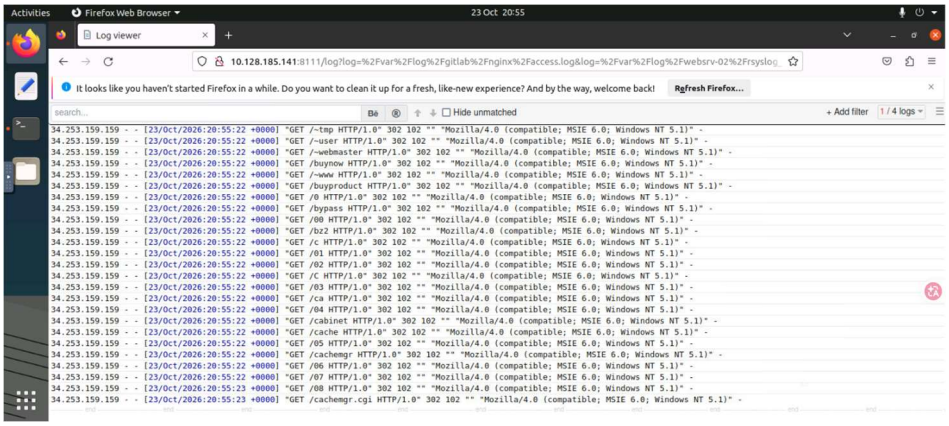


Figure 15. Raw GitLab access log in Log Viewer: one IP requesting many different paths in one second.

> **Analyst assessment**
> Hundreds of different URLs requested in one second from one address, using an outdated user agent, is not human browsing. It matches an automated scan that guesses common paths to find something exposed, which is reconnaissance (MITRE ATT&CK T1595, Active Scanning). This fits the administrator's note about someone persistently scanning the GitLab server.

9.3 Building a consolidated log with awk, sed, sort and uniq

The four logs I needed (the nginx access log, the cron and sshd logs from rsyslog, and the GitLab API log) all use different timestamp styles. I used awk and sed to rewrite each one into a common layout, a timestamp followed by the source file and the original line, and appended every result with >> into one file, /tmp/parsed_consolidated.log.

```
damianhall@ip-10-128-185-141:~$ # Process nginx access log
damianhall@ip-10-128-185-141:~$ awk -F'[][]' '{print "[" $2 "]", "--- /var/log/gitlab/nginx/access
.log ---", "\"" $0 "\""}' /var/log/gitlab/nginx/access.log  | sed "s/ +0000//g" > /tmp/parsed_cons
olidated.log
damianhall@ip-10-128-185-141:~$
damianhall@ip-10-128-185-141:~$ # Process rsyslog_cron.log
damianhall@ip-10-128-185-141:~$ awk '{ original_line = $0; gsub(/ /, "/", $1); printf "[%s/%s/2023
:%s] --- /var/log/websrv-02/rsyslog_cron.log --- \"%s\"\n", $2, $1, $3, original_line }' /var/log/
websrv-02/rsyslog_cron.log >> /tmp/parsed_consolidated.log
damianhall@ip-10-128-185-141:~$
damianhall@ip-10-128-185-141:~$ # Process rsyslog_sshd.log
damianhall@ip-10-128-185-141:~$ awk '{ original_line = $0; gsub(/ /, "/", $1); printf "[%s/%s/2023
:%s] --- /var/log/websrv-02/rsyslog_sshd.log --- \"%s\"\n", $2, $1, $3, original_line }' /var/log/
websrv-02/rsyslog_sshd.log >> /tmp/parsed_consolidated.log
damianhall@ip-10-128-185-141:~$
damianhall@ip-10-128-185-141:~$ # Process gitlab-rails/api_json.log
damianhall@ip-10-128-185-141:~$ awk -F'"' '{timestamp = $4; converted = strftime("[%d/%b/%Y:%H:%M:
%S]", mktime(substr(timestamp, 1, 4) " " substr(timestamp, 6, 2) " " substr(timestamp, 9, 2) " " s
ubstr(timestamp, 12, 2) " " substr(timestamp, 15, 2) " " substr(timestamp, 18, 2) " 0 0")); print
converted, "--- /var/log/gitlab/gitlab-rails/api_json.log ---", "\""$0"\""}' /var/log/gitlab/gitla
b-rails/api_json.log >> /tmp/parsed_consolidated.log
damianhall@ip-10-128-185-141:~$
```

Figure 16. awk and sed commands normalising four different logs into one file.

I then sorted the file by time and removed duplicate lines:

```
damianhall@ip-10-128-185-141:~$ sort /tmp/parsed_consolidated.log > /tmp/sort_parsed_consolidated.
log
damianhall@ip-10-128-185-141:~$ uniq /tmp/sort_parsed_consolidated.log > /tmp/uniq_sort_parsed_con
solidated.log
damianhall@ip-10-128-185-141:~$
```

Figure 17. sort and uniq applied to the consolidated log.

Tool	Role in the pipeline
awk	Splits each line into fields and rebuilds it in the standard layout.
sed	Edits text with substitutions, here removing the timezone offset so the formats match.
sort	Puts all entries in time order, so events from different logs interleave correctly.
uniq	Removes repeated identical lines, reducing noise.
> and >>	Send each result to a file; >> appends so no earlier source is overwritten.

Finally I opened the consolidated file in Log Viewer. The result is one chronological view where the web requests from the suspicious IP sit beside SSH and cron events, which is what makes correlation possible.

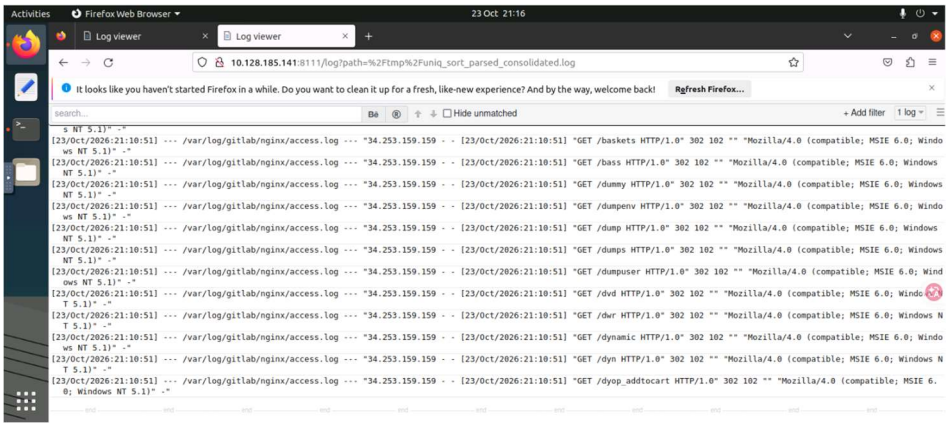


Figure 18. The parsed, sorted and de-duplicated consolidated log in Log Viewer.

9.4 Recommended next steps

- Block or rate-limit 34.253.159.159 at the firewall or web server, and check it against threat intelligence.
- Search the access log for any request from this IP that returned a success code rather than a redirect or error.
- Check the SSH log for the same address, using the grep patterns from section 05.
- Finish configuring rsyslog forwarding to the SIEM so these logs are monitored continuously.

10 Reflection and Key Takeaways

- A handful of commands (grep, tail, awk, sort, uniq) can answer most first-response questions without any special software.
- Generating a failed login myself showed me exactly how it appears in auth.log, and that a loose search term ("Failed") brings false positives that a precise one avoids.
- Knowing where each log lives saves time: authentication problems start in auth.log, web attacks in the web server access log.
- Logs are only useful if they are collected, kept long enough and trusted, which is why rsyslog, logrotate and hashing matter.
- Normalising different logs into one timeline turned scattered lines into a clear story about a single scanning IP.

Next step: practise these searches on live authentication logs and move on to doing the same work inside a SIEM.

11 References

- TryHackMe: Linux Fundamentals Part 1 (free room)
- TryHackMe: Intro to Logs (free room)
- Linux manual pages: grep, tail, awk, sed, rsyslogd, logrotate